

Project Report: Vehicle Parking App V2 (Smart Park)

Name : Preeti Maurya

Roll No. : 23f2003521

Email : 23f2003521@ds.study.iitm.ac.in

About me : I am a BS Diploma level student, with this project I will be completing my Diploma in Programming , with 2 courses of Data science . I am also a final year student at AKTU pursuing Btech in Electrical Engineering . I enjoy learning by building. I believe in combining logic with creativity to solve real-world problems.

Objective

This project involves building a multi-user Vehicle Parking App with separate dashboards for Admin and Users. It manages parking lots and spots, handles real-time booking and releases, and automates background tasks like reminders and reports using Celery. The goal is to implement full-stack development using Flask, VueJS, SQLite, and Redis.

Technologies Used

Flask – Lightweight Python web framework used to build the backend server logic.

Flask-RESTful – Extension for building clean, structured REST APIs with Flask.

Flask-JWT-Extended – Enables secure role-based authentication using JSON Web Tokens.

Flask-SQLAlchemy – ORM for managing database operations using Python objects with SQLite.

Flask-CORS – Allows controlled cross-origin requests between the VueJS frontend and Flask backend.

VueJS – Progressive JavaScript framework used for building interactive, component-based UI.

Bootstrap (via CDN) – For designing responsive and styled frontend layouts.

SQLite – Embedded relational database to persist data like users, lots, and reservations.

ChartJS–For Plotting Graphs and summary visualisation

Redis – In-memory key-value store used for caching and as a message broker for Celery.

Celery – Handles asynchronous background jobs like sending reminders and generating reports.

Google Chat Webhooks / Email (MailHog) – Used for sending scheduled notifications and activity reports to users.

DB Schema Design:

The database consists of the following tables:-

1.Users

Columns	id	username	type	email	password	vehicle_no
Type	Integer	String	String	String	String	Integer
Constraints	Primary Key	Not Null	Default='user'	Unique,Not Null	Not Null	Not Null
Description	Unique Identifier	Display Name	Role	Used for login	Hashed Password	User's Vehicle

2.ParkingLot

Columns	id	prime_location_name	price_per_hour	address	pin_code	number_of_spot	created_at
Type	Integer	String	Integer	String	Integer	Integer	DateTime
Constraints	Primary Key	Not Null	Not Null	Not Null	Not Null	Not Null	Default=Now() ()
Description	Unique Identifier for Lot	Known near Place Name	Cost/hr	Exact location		Spots available at Lot	When Lot was created

3.ParkingSpot

Columns	id	status	lot_id
Type	Integer	String	Integer

Constraints	Primary Key	Default('A')	Not Null
Description	Unique Identifier for Spot	Spot status ('A','O','B')	ForeignKey to Lot

4.Reservation

Columns	id	spot_id	user_id	parking_time	leaving_time	parking_cost	reservation_status
Type	Integer	Integer	Integer	DateTime	DateTime	Float	String
Constraints	Primary Key	Not Null	Not Null	Not Null	Not Null	Not Null	Default('Booked')
Description	Unique Identifier for Reservation	ForeignKey to Spot	ForeignKey to user	Vehicle Parking Time	Vehicle Leaving Time	Total cost to pay	Status of the reservation

API Design:

In this project, RESTful APIs were created to handle the core functionalities of the system and ensure smooth interaction between the frontend and backend. These APIs were built using **Flask** and followed REST standards for clear and scalable communication. The main entities exposed through the API included **Users**, **Parking Lots**, **Bookings**, **Reservations**, and **Admin Operations**.

Architecture Overview and Features

The project follows an MVC-like architecture split into frontend and backend layers.

- **Frontend**
 - Built with Vue.js and organized under `src/components/`, it includes components like `Login.vue`, `Register.vue`, `Dashboard.vue`, `AdminSummary.vue`, etc. Routing is managed via `routes.js`, with global styles in `Style1.css`
- **Backend** (`backend/`): Developed in Flask.
 - `controllers.py`: All API endpoints are defined here.
 - `models.py`: SQLAlchemy ORM models for users, lots, reservations, etc.
 - `tasks.py`: Celery tasks for background operations like overstay detection.
 - `email.py`, `security.py`: Handle email alerts and JWT-based auth.
 - `config.py`: Environment and app settings.
 - `celery.py`: Initializes Celery app.
- **Static Files**: `static/` holds report CSVs, images.
- **Database**: SQLite DB located at `instance/vpapp0.sqlite3`.
- **Entry Point**: `app.py` initializes Flask app

System WorkFlow:

- Authentication – Users (Admin/User) register and log in using JWT-based APIs; redirected to role-specific dashboards after login
- User Operations – Search lots, view spot statuses, book/release spots, download booking history (CSV).
- Admin Operations – Add/update/delete lots and spots, search users/lots, view lot summaries..
- Automation (Celery) – Periodically updates spot statuses, detects overstays, applies penalties, sends alerts and reports to email
- Profile Management – Users can view profile info and gain insights.

Use of AI/LLM : AI tools, including ChatGPT, were used to assist in understanding concepts, debugging errors, and refining documentation. However, the core logic, API design, database schema, and frontend implementation were written and structured by me. The AI was used as a learning aid and code reviewer, not as a code generator for complete modules.

Video Link: [Video](#)